Práctica 1

Jordi Navarro Salas
48764359N

10 de octubre de 2026

Código fuente:
`https://github.com/jordiinavarroo/sistemasoperativos/tree/main`

Índice general

1 Ejercicio 1A **1**
1.1 Enunciado del Ejercicio 1
1.2 Identificación y Justificación de Tareas 1
1.3 Diagrama de Secuencia 2
1.4 Análisis Detallado del Código Fuente 2

2 Ejercicio 1B **6**
2.1 Enunciado del Ejercicio 6
2.2 Identificación y Justificación de Tareas 6
2.3 Diagrama de Secuencia 7
2.4 Análisis Detallado del Código Fuente 7

3 Ejercicio 2 **9**
3.1 Enunciado del Ejercicio 9
3.2 Identificación y Justificación de Tareas 9
3.3 Diagrama de Secuencia 9
3.4 Análisis Detallado del Código Fuente 10

4 Ejercicio 3 **12**
4.1 Enunciado del Ejercicio 12
4.2 Identificación y Justificación de Tareas 12
4.3 Diagrama de Secuencia 13
4.4 Análisis Detallado del Código Fuente 13

Capítulo 1

Ejercicio 1A

1.1 Enunciado del Ejercicio

Realiza un programa llamado `ejec.c` que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.1.1 Estructura del Árbol de Procesos

La jerarquía de procesos que se debe crear es la siguiente:

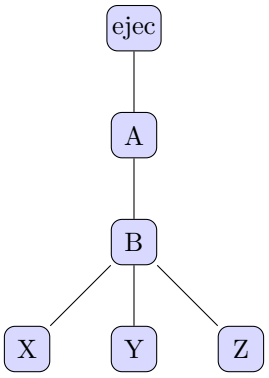


Figura 1.1: Diagrama del árbol de procesos a generar.

1.1.2 Resultado Esperado

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752...

/* Tras un intervalo de 15 segundos aparecera */
/* Resultado del comando pstree */

Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

1.2 Identificación y Justificación de Tareas

Recepción de Argumentos. La función `parse_args()` comprueba que se reciba un solo argumento, lo convierte a entero y lo guarda en la variable global `g_seconds`. Si no es un número positivo, el programa termina.

Creación del Árbol de Procesos. Para crear la jerarquía `ejec -> A -> B -> \{X,Y,Z\}` cada proceso tiene su propia función (`create_A_process()`, `create_B_process()`, etc.) que hace el `fork()`. Los pids se guardan en variables globales para que los procesos puedan mandarse señales entre ellos.

Alarma y Ejecución de pstree. Como Z no puede usar `sleep()`, programa una alarma con `alarm()` y espera con `pause()`. Cuando llega `SIGALRM`, manda `SIGUSR1` a A con `kill()`. A crea un hijo que ejecuta `pstree -c` con `execlp()` (si lo hiciera A directamente, dejaría de ser A) y espera con `wait()` a que termine.

Destrucción Controlada del Árbol. Para que los padres no mueran antes que los hijos, cuando acaba `pstree` A avisa a `ejec` con `SIGUSR2` y `ejec` se la reenvía. De ahí va a A, luego a B, y B se la manda a Z, Y y X haciendo `wait()` después de cada una. Así el orden de muerte es Z, Y, X, B, A y `ejec`.

1.3 Diagrama de Secuencia

El siguiente diagrama muestra el ciclo de vida del programa: la creación del árbol, la alarma de Z, la ejecución de `pstree` y la destrucción controlada.

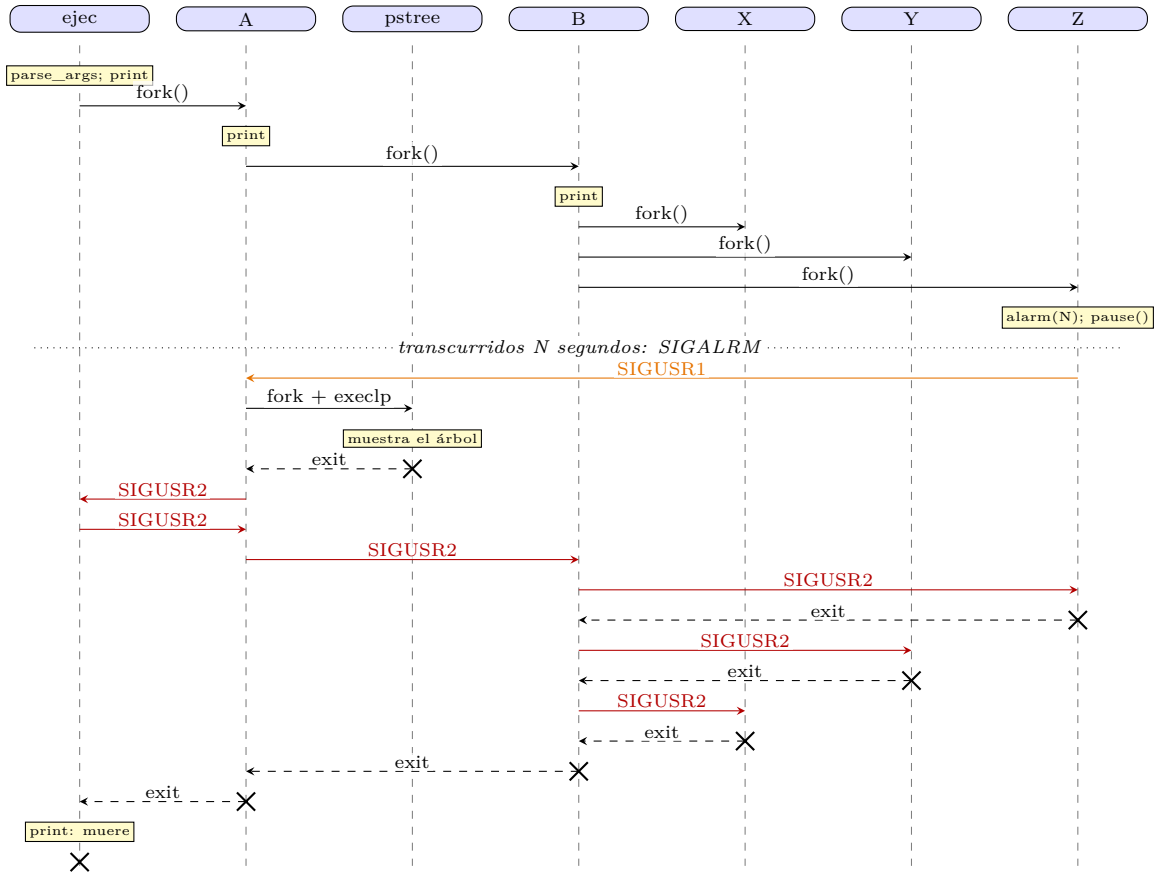


Figura 1.2: Diagrama de secuencia de `ejec.c`.

1.4 Análisis Detallado del Código Fuente

Funciones `parse_args()` y `main()`

`parse_args()` comprueba que haya un solo argumento y que sea un número positivo. `main()` guarda el pid de `ejec`, instala el manejador de `SIGUSR2` y crea a A. Cuando `create_A_process()` termina, es que ya han muerto todos los demás y `ejec` imprime que muere. `handler_start_destruction()` reenvía `SIGUSR2` a A para empezar la destrucción.

Llamadas al Sistema Utilizadas: `exit()`, `getpid()`, `signal()`, `wait()`, `kill()`.

```c
void parse_args(int argc, char *argv[]) {
    if (argc != 2) {
        exit(EXIT_FAILURE);
    }
    g_seconds = atoi(argv[1]);
    if (g_seconds <= 0) {
        exit(EXIT_FAILURE);
    }
}

int main(int argc, char *argv[]) {
    parse_args(argc, argv);
    pid_arb = getpid();
    printf("Soy el proceso ejec: mi pid es %d\n", pid_arb);

    signal(SIGUSR2, handler_start_destruction);
    create_A_process();

    printf("Soy ejec (%d) y muero\n", pid_arb);
    return 0;
}

void handler_start_destruction(int s) {
    kill(pid_a, SIGUSR2);
}
```

Proceso A

`create_A_process()` crea a A y espera a que termine toda su rama. A instala sus manejadores y crea a B. Cuando recibe `SIGUSR1` de Z, crea un hijo que ejecuta `pstree -c`, espera a que acabe, avisa a `ejec` con `SIGUSR2` y hace `pause()`. Cuando la señal le vuelve, se la manda a B.

Llamadas al Sistema Utilizadas: `fork()`, `wait()`, `getpid()`, `signal()`, `execlp()`, `kill()`, `pause()`.

```c
void create_A_process() {
    switch (pid_a = fork()) {
        case -1:
            perror("fork A");
            exit(EXIT_FAILURE);
        case 0:
            run_process_A();
            exit(0);
        default:
            wait(NULL);
    }
}

void run_process_A() {
    pid_a = getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", pid_a, pid_arb);
    signal(SIGUSR1, handler_A_exec_task);
    signal(SIGUSR2, handler_A_destroy_and_propagate);
    create_B_process();
    printf("Soy A (%d) y muero\n", pid_a);
}

void handler_A_exec_task(int s) {
    char arb_cad[100];
    sprintf(arb_cad, "%d", pid_arb);
    if(fork() == 0){
        execlp("pstree", "pstree", "-c", arb_cad, NULL);
    }
    else{
        wait(NULL);
        kill(pid_arb, SIGUSR2);
        pause();
    }
}

void handler_A_destroy_and_propagate(int s) {
    kill(pid_b, SIGUSR2);
}
```

Proceso B

B imprime su información, instala el manejador de `SIGUSR2` y crea a X, Y y Z con un `usleep()` de 50 ms entre cada uno para que los mensajes salgan siempre en el mismo orden. Después hace `pause()`. Cuando recibe la señal, se la manda a Z, Y y X y espera con `wait()` a que muera cada uno antes de pasar al siguiente.

Llamadas al Sistema Utilizadas: `fork()`, `wait()`, `getpid()`, `signal()`, `usleep()`, `pause()`, `kill()`.

```c
void create_B_process() {
    switch (pid_b = fork()) {
        case -1:
            perror("fork B");
            exit(EXIT_FAILURE);
        case 0:
            run_process_B();
            exit(0);
        default:
            wait(NULL);
    }
}

void run_process_B() {
    pid_b = getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n",
           pid_b, pid_a, pid_arb);
    signal(SIGUSR1, SIG_DFL);
    signal(SIGUSR2, handler_B_destroy_and_propagate);

    create_X_process();
    usleep(50000);
    create_Y_process();
    usleep(50000);
    create_Z_process();

    pause();
    printf("Soy B (%d) y muero\n", pid_b);
}

void handler_B_destroy_and_propagate(int s) {
    kill(pid_z, SIGUSR2);
    wait(NULL);
    kill(pid_y, SIGUSR2);
    wait(NULL);
    kill(pid_x, SIGUSR2);
    wait(NULL);
}
```

Procesos X e Y

X e Y hacen lo mismo: imprimen su información, esperan con `pause()` y, cuando reciben `SIGUSR2`, imprimen que mueren y terminan. Solo se enseña el código de X porque el de Y es igual cambiando la letra.

Llamadas al Sistema Utilizadas: `fork()`, `getpid()`, `signal()`, `pause()`, `exit()`.

```c
void create_X_process() {
    switch (pid_x = fork()) {
        case -1:
            perror("fork X");
            exit(EXIT_FAILURE);
        case 0:
            run_process_X();
            exit(0);
        default:
            break;
    }
}

void run_process_X() {
    pid_x = getpid();
    printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es %d. "
           "Mi bisabuelo es %d\n", pid_x, pid_b, pid_a, pid_arb);
    signal(SIGUSR2, handler_X_destroy_leaf);
    pause();
}

void handler_X_destroy_leaf(int s) {
    printf("Soy X (%d) y muero\n", getpid());
    exit(0);
}
```

Proceso Z

Z instala los manejadores de `SIGALRM` y `SIGUSR2`, programa la alarma con los segundos del argumento y espera con `pause()` dentro de un bucle (sin usar `sleep()`). Cuando salta la alarma manda `SIGUSR1` a A, y cuando recibe `SIGUSR2` imprime que muere y termina.

Llamadas al Sistema Utilizadas: `fork()`, `getpid()`, `signal()`, `alarm()`, `pause()`, `kill()`, `exit()`.

```c
void create_Z_process() {
    switch (pid_z = fork()) {
        case -1:
```

```
4               perror("fork Z");
5               exit(EXIT_FAILURE);
6           case 0:
7               run_process_Z();
8               exit(0);
9           default:
10              break;
11      }
12  }
13
14  void run_process_Z() {
15      pid_z = getpid();
16      printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", pid_z,
            pid_b, pid_a, pid_arb);
17      signal(SIGALRM, handler_Z_alarm);
18      signal(SIGUSR2, handler_Z_destroy_leaf);
19      alarm(g_seconds);
20      while(1){
21          pause();
22      }
23  }
24
25  void handler_Z_alarm(int s) {
26      kill(pid_a, SIGUSR1);
27  }
28
29  void handler_Z_destroy_leaf(int s) {
30      printf("Soy Z (%d) y muero\n", getpid());
31      exit(0);
32  }
```

Capítulo 2

Ejercicio 1B

2.1 Enunciado del Ejercicio

Realiza un programa llamado `malla.c` que produzca el árbol de procesos que se muestra a continuación. El programa recibirá dos argumentos `x` e `y` que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente se está creando se debe emplear la llamada al sistema "pstree -c".

2.1.1 Estructura del Árbol de Procesos

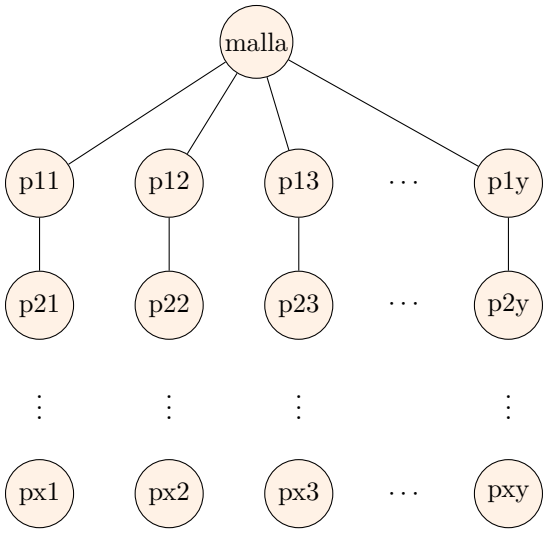


Figura 2.1: Árbol de procesos a generar (x filas e y columnas).

2.1.2 Resultado Esperado

Para $x = 3$ e $y = 2$ (tres filas y dos columnas), `pstree -c` debe mostrar dos cadenas de tres procesos colgando de `malla`:

```
$ malla 3 2 &
$ pstree -c <pid>
malla-+-malla---malla---malla
      `-malla---malla---malla
```

2.2 Identificación y Justificación de Tareas

Recepción de Argumentos. La función `parse_args()` comprueba que se reciban dos argumentos, los convierte a entero y mira que los dos sean positivos. Si no, el programa termina.

Creación de la Malla. El árbol son `y` columnas con `x` procesos cada una. Se hace con dos bucles: el exterior crea un hijo por cada columna y cada uno de esos hijos usa el bucle interior para ir creando su cadena de `x` procesos.

Hojas y Temporización. El último proceso de cada columna es la hoja. Para poder ver el árbol con `pstree -c` tiene que seguir vivo un rato, así que cada hoja hace `alarm(10)` y `pause()`. Como la acción por defecto de `SIGALRM` es matar al proceso, se pone un manejador vacío para que `pause()` termine y ya está.

Terminación Ordenada. Para que los padres no mueran antes que los hijos, cada proceso que ha creado un hijo hace `wait()` antes de acabar, y el proceso principal espera a sus `y` hijos. Así el árbol se destruye de abajo hacia arriba.

2.3 Diagrama de Secuencia

El siguiente diagrama muestra la creación y la terminación de los procesos para $x = 3$ e $y = 2$. Los nombres son los de la figura 2.1.

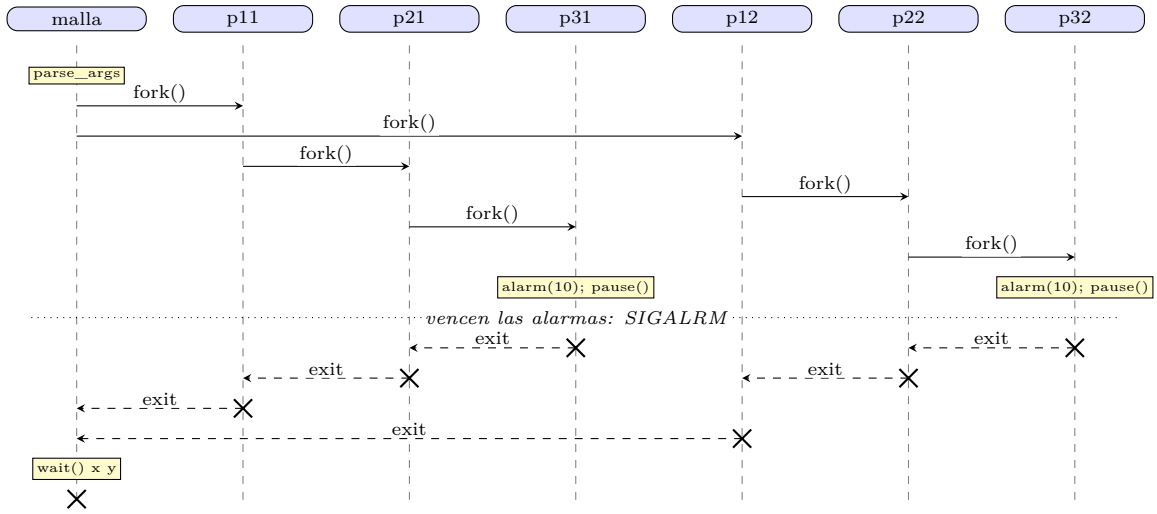


Figura 2.2: Diagrama de secuencia de `malla.c` ($x = 3$, $y = 2$).

2.4 Análisis Detallado del Código Fuente

Funciones de apoyo

`parse_args()` comprueba los dos argumentos y los guarda en `g_x` y `g_y`. `handler_leaf_alarm()` es un manejador vacío de `SIGALRM` para que la hoja no muera cuando salta la alarma. `create_process()` hace el `fork()` y comprueba que no falle. `close_process()` espera al hijo y termina. `run_leaf_process()` programa la alarma de 10 segundos de las hojas.

Llamadas al Sistema Utilizadas: `fork()`, `wait()`, `exit()`, `signal()`, `alarm()`, `pause()`.

```
1  void parse_args(int argc, char *argv[]) {
2      if (argc != 3) {
3          exit(EXIT_FAILURE);
4      }
5      g_x = atoi(argv[1]);
6      g_y = atoi(argv[2]);
7      if (g_x <= 0 || g_y <= 0) {
8          exit(EXIT_FAILURE);
9      }
10 }
11
12 void handler_leaf_alarm(int s) {
13 }
14
15 pid_t create_process() {
16     pid_t pid = fork();
17     if (pid == -1) {
18         perror("fork");
19         exit(EXIT_FAILURE);
20     }
21     return pid;
22 }
23
24 void close_process() {
25     wait(NULL);
26     exit(0);
27 }
28
29 void run_leaf_process() {
```

```
30      signal(SIGALRM, handler_leaf_alarm);
31      alarm(10);
32      pause();
33  }
```

Función `main()`

El bucle de `i` crea un hijo por cada columna. Cada hijo entra en el bucle de `j` y va creando un hijo en cada vuelta: el que acaba de crear un hijo hace `close_process()` (espera y termina) y el hijo sigue con el bucle. El último de la cadena (`j == g_x`) ejecuta `run_leaf_process()`. Al final, el proceso principal, que es el único que llega a `i == g_y + 1`, espera a sus `y` hijos.

Llamadas al Sistema Utilizadas: `wait()`.

```
1  int main(int argc, char *argv[]) {
2      pid_t pid;
3      int i, j;
4
5      parse_args(argc, argv);
6
7      for (i = 1; i <= g_y; i++) {
8          pid = create_process();
9          if (pid == 0) {
10             for (j = 1; j <= g_x - 1; j++) {
11                 pid = create_process();
12                 if (pid != 0) {
13                     close_process();
14                 }
15             }
16             if (j == g_x) {
17                 run_leaf_process();
18                 close_process();
19             }
20         }
21     }
22     if (i == g_y + 1) {
23         for (i = 1; i <= g_y; i++) {
24             wait(NULL);
25         }
26     }
27     return 0;
28 };
```

Capítulo 3

Ejercicio 2

3.1 Enunciado del Ejercicio

Realizar un programa llamado `hacha.c` que divida un archivo en varios trozos con el mismo nombre y extensión `h00`, `h01`, … Para dividir, el proceso `hacha` (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos. Las entradas y salidas a los archivos se realizarán con llamadas al sistema (no se puede utilizar `printf`, `scanf`, etc.), y es decisión del alumno si los hijos se lanzan secuencial o concurrentemente.

3.1.1 Resultado Esperado

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

3.1.2 Esquema de Funcionamiento

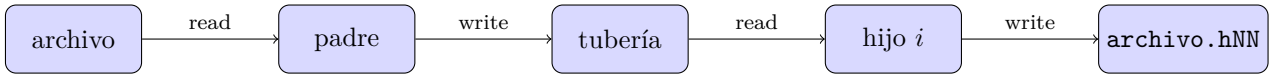


Figura 3.1: Recorrido de los datos de cada trozo.

3.2 Identificación y Justificación de Tareas

Recepción de Argumentos. El programa se inicializa con `hacha <archivo> <tamaño>`. La función `main()` comprueba que se reciban exactamente dos argumentos, convierte el tamaño a entero con `atoi()` y llama a la función `slice()`.

Cálculo del Número de Trozos. Se obtiene el tamaño del archivo con `stat()` y se divide entre el tamaño del trozo. Si la división no es exacta se incrementa el contador.

Creación de los Archivos de Salida. Cada hijo construye el nombre del archivo destino añadiendo `.h00`, `.h01`, etc. lo crea con `creat()`, lee el trozo de la tubería y lo escribe en el archivo.

Llamadas al Sistema y Terminación. Toda la entrada y salida de archivos se realiza con `open()`, `read()`, `write()`, `creat()` y `close()`, sin usar `printf` ni `scanf`. Al finalizar, el padre libera la memoria reservada y espera a todos sus hijos con `wait()`.

3.3 Diagrama de Secuencia

El siguiente diagrama muestra el recorrido de los datos para un archivo que se divide en dos trozos.

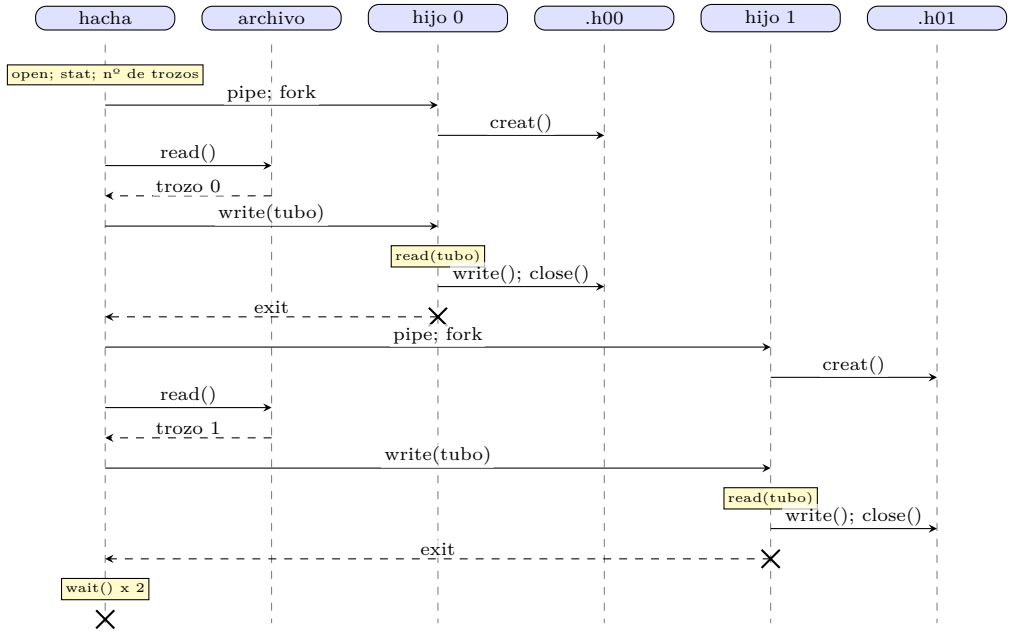


Figura 3.2: Diagrama de secuencia de `hacha.c` (dos trozos).

3.4 Análisis Detallado del Código Fuente

Función `main()`

Comprueba que se reciban dos argumentos y llama a `slice()` con el nombre del archivo y el tamaño de trozo.

Llamadas al Sistema Utilizadas: `atoi()`.

```
1 int main(int argc, char *argv[]) {
2     if(argc == 3) {
3         slice(argv[1], atoi(argv[2]));
4     }
5     return 0;
6 }
```

Función `slice()`

Primero reserva un buffer del tamaño de un trozo y abre el archivo de entrada con `open()`. Con `stat()` calcula el número de trozos. Después, en el bucle, cada iteración del bucle for crea una tubería y un hijo. El padre lee un trozo del archivo y lo escribe en la tubería. El hijo forma el nombre del archivo de salida, lo crea, lee de la tubería, escribe en el archivo, lo cierra, libera el buffer y sale del bucle. Al terminar el bucle, solo el padre cumple `i == slices`, libera su buffer y espera a todos los hijos con `wait()`.

Llamadas al Sistema Utilizadas: `open()`, `stat()`, `pipe()`, `fork()`, `read()`, `write()`, `creat()`, `close()`, `wait()`.

```
1  void slice(char name[], int size) {
2      int i, slices, slicesRead, fileIn, fileOut;
3      char *sliceRead;
4      int tube[2];
5      struct stat fileStats;
6      char file[50];
7      sliceRead = (char *) malloc(sizeof(char) * size);
8      fileIn = open(name, O_RDONLY);
9      if(fileIn == 0) {
10         stat(name, &fileStats);
11         slices = fileStats.st_size / size;
12         if(fileStats.st_size % size != 0) {
13             slices++;
14         }
15         for(i = 0; i < slices; i++) {
16             pipe(tube);
17             if(fork() != 0) {
18                 slicesRead = read(fileIn, sliceRead, size);
19                 write(tube[1], sliceRead, slicesRead);
```

```
20              } else {
21                  if(i < 10) {
22                      sprintf(file, "%s.h0%d", name, i);
23                  } else {
24                      sprintf(file, "%s.h%d", name, i);
25                  }
26                  fileOut = creat(file, 0666);
27                  slicesRead = read(tube[0], sliceRead, size);
28                  write(fileOut, sliceRead, slicesRead);
29                  close(fileOut);
30                  free(sliceRead);
31                  break;
32              }
33          }
34          if(i == slices) {
35              free(sliceRead);
36              for(int i = 0; i < slices; i++) {
37                  wait(NULL);
38              }
39          }
40      }
41  }
```

Capítulo 4

Ejercicio 3

4.1 Enunciado del Ejercicio

Diseña un programa llamado `hijos.c` que cree un árbol de procesos según la estructura que se indica a partir de dos parámetros: `hijos x y`. El proceso `hijos` debe mostrar el mensaje *"Soy el superpadre (pid): mis hijos finales son: pidx1, pidx2, …, pidxy"*. Los procesos de la parte baja deben mostrar el mensaje *"Soy el subhijo pidx1, mi padres son: pid1, pid2, …, pidx"*. Para que los procesos puedan compartir los pids se debe utilizar memoria compartida.

4.1.1 Estructura del Árbol de Procesos

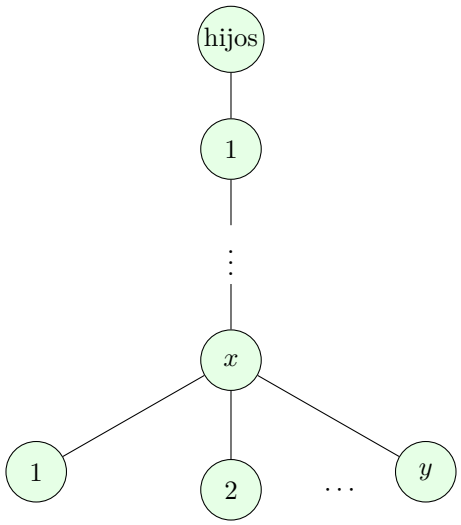


Figura 4.1: Cadena vertical de x procesos con y hijos finales.

4.1.2 Resultado Esperado

Ejemplo para `hijos 3 2`:

```
$ hijos 3 2
Soy el proceso 435. Mis padres son: 434
Soy el proceso 436. Mis padres son: 434, 435
Soy el proceso 437. Mis padres son: 434, 435, 436
Soy el super padre 434, mis hijos finales son: 438, 439
```

4.2 Identificación y Justificación de Tareas

Segmentos de Memoria Compartida. Se crean dos segmentos con `shmget()` usando `IPC_PRIVATE` y se vinculan con `shmat()`. El primero (`px`) guarda los `x` pids de la cadena y el segundo (`py`) los `y` pids de los hijos finales.

Cadena Vertical. Un bucle crea los `x` procesos de la cadena: en cada iteración del bucle for el padre espera con `wait()` y el hijo guarda su pid en `px` e imprime sus padres.

Hijos Finales. El último proceso de la cadena crea los `y` hijos finales. Cada uno guarda su pid en `py`, espera 10 segundos con `pause()` para poder observar el árbol con `pstree`.

Liberación de la Memoria Compartida. Cuando todo ha terminado, el padre imprime los pids de los hijos, y los elimina.

4.3 Diagrama de Secuencia

El siguiente diagrama muestra la ejecución para $x = 3$ procesos en la cadena vertical e $y = 2$ hijos finales (`f1` y `f2`); los procesos `1`, `2` y `3` son los de la cadena.

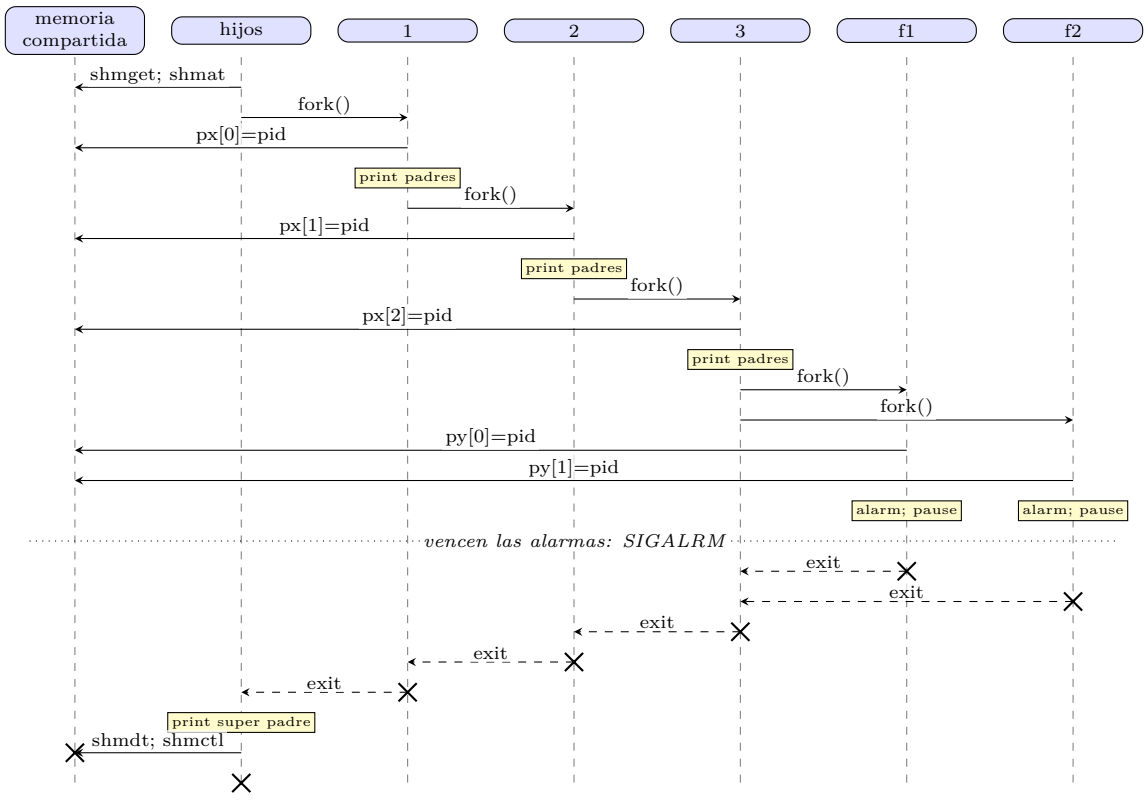


Figura 4.2: Diagrama de secuencia de `hijos.c` ($x = 3$, $y = 2$).

4.4 Análisis Detallado del Código Fuente

El programa se muestra completo y se explica por bloques.

```
9  void end(){};
10
11 int main(int argc, char *argv[]){
12     if(argc == 3){
13         int i;
14         pid_t pid;
15         pid_t pid_desc = getpid();
16         int x = atoi(argv[1]);
17         int y = atoi(argv[2]);
18         int shmx = shmget(IPC_PRIVATE, sizeof(int) * x, IPC_CREAT | 0666);
19         int shmy = shmget(IPC_PRIVATE, sizeof(int) * y, IPC_CREAT | 0666);
20         int *px = (int *) shmat (shmx, 0, 0);
21         int *py = (int *) shmat (shmy, 0, 0);
22
23         for(i = 1; i <= x; i++){
24             pid = fork();
25             if(pid != 0){
26                 wait(NULL);
27                 break;
28             }
29             else{
30                 px[i - 1] = getpid();
31                 printf("Soy el proceso %d. Mis padres son: ", getpid());
32                 printf("%d", pid_desc);
33                 for(int j = 0; j < i - 1; j++){
34                     printf(", %d", px[j]);
35                 }
```

```
36                  printf("\n");
37              }
38          }
39          if(i == 1){
40              printf("Soy el super padre %d, mis hijos finales son: ", getpid());
41              for(i = 0; i < y; i++){
42                  printf("%d" ,py[i]);
43                  if(i != y - 1){
44                      printf(", ");
45                  }
46              }
47              printf("\n");
48              shmdt((char *) px);
49              shmdt((char *) py);
50              shmctl(shmx, IPC_RMID, 0);
51              shmctl(shmy, IPC_RMID, 0);
52          }
53          else{
54              if(i == x + 1){
55                  for(i = 1; i <= y; i++){
56                      pid = fork();
57                      if(pid == 0){
58                          py[i-1] = getpid();
59                          signal(SIGALRM, end);
60                          alarm(10);
61                          pause();
62                          break;
63                      }
64                  }
65                  if(i == y + 1){
66                      for(i = 1; i <= y; i++){
67                          wait(NULL);
68                      }
69                  }
70              }
71          }
72      };
73      return 0;
74  }
```